Universitatea Națională de Știință și Tehnologie
Politehnica București
Facultatea de Științe Aplicate
2026

Proiect Teoria Jocurilor

Duelul Strategic

Facultatea de Științe Aplicate
Anul III, grupa 1333, semigr. B
Cercetări Operaționale - Proiect

Studenţi:

Stroescu Adrian-Gabriel

Tînjală Bianca-Ionela

Vlad Adrian

Volosenco Andreea-Laura

Cuprins:

1. Introducere în Teoria Jocurilor

1.1. Obiectul de studiu și rolul procesului decizional

1.2. Structura și tipologia jocurilor: criterii de clasificare și terminologie

2. Modelarea Jocurilor Matriceale cu Sumă Nulă

2.1. Formalizarea matematică a jocurilor de două persoane: tripletul (X, Y, f)

2.2. Reprezentarea prin matricea câștigurilor și semnificația elementelor a_{ij}

2.3. Analiza strategiilor pure și a variantelor de acțiune

3. Criterii de Optimalitate și Echilibrul Jocului

3.1. Principiile Max-Min și Min-Max: determinarea valorilor v_1 și v_2

3.2. Punctul de Șa (Saddle Point): condiții de existență și stabilitate

3.3. Teorema lui Nash: generalizarea echilibrului în sistemele decizionale

4. Relația dintre Teoria Jocurilor și Programarea Liniară

4.1. Transpunerea jocurilor de strategie în modele de optimizare liniară

4.2. Dualitatea în contextul jocurilor: relația dintre strategiile jucătorilor

4.3. Strategii de calcul numeric și selecția algoritmului optim

5. Implementarea Computațională a Modelului

5.1 Implementarea algoritmului

5.2 Codul Sursa

5.3. Schema logica

6. Aplicație Practică: "Duelul Strategic"

6.1. Construcția modelului decizional și a matricei de câștig

6.2. Determinarea soluțiilor optime și interpretarea rezultatelor

7. Concluzii

8. Bibliografie

1. Introducere în Teoria Jocurilor

1.1. Obiectul de studiu și rolul procesului decizional

Teoria jocurilor reprezintă o disciplină fundamentală în cadrul cercetărilor operaționale, având ca obiectiv principal optimizarea procesului de adoptare a deciziilor în situații caracterizate prin interacțiune strategică, competiție sau conflict. În termeni inginerești, jocul este definit ca o succesiune de decizii atribuite unor factori conștienți, numiți jucători, care operează într-un sistem guvernat de reguli stricte.

Obiectul de studiu al acestei discipline constă în analiza ansamblului de jucători și a acțiunilor acestora, orientate către obținerea unui rezultat cuantificabil (câștig sau pierdere) la finalul unei interacțiuni. Rolul procesului decizional este esențial, deoarece fiecare participant urmărește atingerea unui obiectiv optim:

- Maximizarea beneficiului: obținerea unui câștig maxim în condiții de concurență.
- Minimizarea costului: reducerea pierderilor la o valoare minimă posibilă.

Complexitatea procesului decizional în teoria jocurilor derivă din natura informației de care dispune decidentul. Un jucător poate adopta decizii bazându-se pe:

1. Informații complete: cunoașterea integrală a regulilor și a posibilităților de acțiune ale adversarului.
2. Informații parțiale sau lipsa acestora: situații în care acțiunile adversarilor nu pot fi prevăzute cu exactitate.
3. Influența factorilor aleatori: intervenția hazardului sau a unor elemente de probabilitate care pot influența rezultatul final.

În acest context, adoptarea unei decizii nu este un act izolat, ci rezultatul unei analize a mulțimii de alternative disponibile, numite strategii. Procesul decizional presupune alegerea acelei strategii care să asigure stabilitatea și eficiența soluției, indiferent de variabilitatea acțiunilor celorlalți participanți la joc.

1.2. Structura și tipologia jocurilor: criterii de clasificare și terminologie

Un joc este reprezentat printr-un ansamblu de jucători (participanți sau competitori) care se supun aceloraşi reguli, acțiunile acestora fiind orientate către adoptarea unei decizii finale. Elementele structurale de bază includ:

- Jucătorii: factorii decizionali care intră în competiție.
- Strategiile: mulțimea de alternative de care dispune fiecare jucător pentru a adopta o decizie.
- Rezultatul: atribuirea unui câștig sau a unei pierderi fiecărui participant la finalul jocului.

Din punct de vedere al tipologiei, jocurile se clasifică în funcție de criterii precise care definesc modul de interacțiune și natura sistemului:

1. După numărul de jucători:
    - Jocuri de 2 persoane.
    - Jocuri de $n > 2$ persoane.
    - Jocuri de o persoană (jocuri contra naturii), unde cea de-a doua entitate este mediul înconjurător.
2. După natura împărțirii câștigurilor:
    - Jocuri cu sumă nulă: suma algebrică a câștigurilor și pierderilor tuturor jucătorilor este egală cu zero ($P_k = 0$).
    - Jocuri fără sumă nulă: suma câștigurilor este diferită de zero.
3. După mulțimea strategiilor:
    - Jocuri finite: mulțimea strategiilor este limitată.
    - Jocuri infinite: mulțimea strategiilor este nelimitată.
4. După gradul de cooperare:
    - Jocuri cooperative: permit formarea de coaliții între jucători.
    - Jocuri necooperative: nu permit înțelegeri prealabile sau plăți parțiale între participanți.
5. După nivelul de informare:
    - Jocuri cu informație completă: jucătorul deține date integrale despre modul de acțiune și regulile sistemului.
    - Jocuri cu informație incompletă: jucătorul nu poate prevedea cu exactitate strategiile adoptate de adversar.
    - Jocuri de hazard: situații în care intervin exclusiv factori aleatori.

2. Modelarea Jocurilor Matriceale cu Sumă Nulă

2.1. Formalizarea matematică a jocurilor de două persoane: tripletul (X, Y, f)

În cadrul modelării analitice, un joc de două persoane (notate convențional A și B), cu sumă nulă și în formă normală, este reprezentat formal printr-un ansamblu ordonat de tipul (X, Y, f). Această structură abstractizează realitatea conflictuală, transformând deciziile subiective în variabile matematice prelucrabile.

Componentele acestui model sunt definite după cum urmează:

1. Mulțimea strategiilor jucătorului A (X):
   - Reprezintă spațiul stărilor sau totalitatea alternativelor de care dispune primul decident.
   - În modelul finit, aceasta este o mulțime discretă $X = \{a_1, a_2, ..., a_m\}$, unde fiecare element a_i este o variantă de acțiune menită să conducă la atingerea obiectivului.
   - Din punct de vedere ingineresc, X poate fi privit ca un set de configurații de sistem sau parametri de intrare controlați de primul agent.
2. Mulțimea strategiilor jucătorului B (Y):
   - Reprezintă ansamblul de răspunsuri sau contrastratégii disponibile pentru cel de-al doilea jucător (adversarul sau natura).
   - Pentru jocurile finite, mulțimea este definită ca $Y = \{b_1, b_2, ..., b_n\}$.
   - Existența acestor două mulțimi, X și Y, ambele nevide ($X, Y \neq \phi$), este condiția obligatorie pentru existența interacțiunii strategice.
3. Funcția de câștig (f):
   - Este o funcție scalară definită pe produsul cartezian al celor două mulțimi, $f : X \times Y \to \mathbb{R}$.
   - Aceasta mapează fiecare pereche de strategii (a_i, b_j) într-o valoare reală a_{ij}, numită plată sau rezultat.
   - Valoarea $f(a_i, b_j) = a_{ij}$ reprezintă beneficiul net al jucătorului A. Datorită naturii de sumă nulă a jocului, câștigul lui A este echivalent cu pierderea lui B.

Interdependența decizională și optimizarea Relația fundamentală care guvernează acest triplet este $f_A + f_B = 0$, ceea ce implică $f_B = -f_A$. Din această perspectivă, modelul (X, Y, f) devine un sistem de optimizare cu obiective opuse:

- Jucătorul A acționează ca un operator de maximizare, încercând să aleagă $x \in X$ astfel încât să obțină un beneficiu maxim.
- Jucătorul B acționează ca un operator de minimizare, alegând $y \in Y$ pentru a limita pierderea la o valoare minimă.

Această formalizare permite transformarea oricărei probleme de decizie într-o analiză matriceală, unde obiectivul este găsirea unui punct de echilibru în care strategiile alese de ambii jucători sunt optime în raport cu funcția de câștig f.

2.2. Reprezentarea prin matricea câștigurilor și semnificația elementelor a_{ij}

Transpunerea modelului analitic într-o formă de calcul procesabilă se realizează prin construcția matricei jocului, notată în schițele de proiect și în literatura de specialitate cu Q. Această matrice reprezintă forma normală a jocului și servește ca bază pentru aplicarea algoritmilor de optimizare (Simplex sau căutarea punctului de șa).

Matricea Q este un tablou bidimensional care corelează strategiile celor doi jucători, A (jucătorul care urmărește câștigul) și B (jucătorul care urmărește minimizarea pierderii). Structura generală a unui joc matriceal este prezentată în Tabelul 1

A \ B	b_1	b_2	$\dots$	b_n	$\alpha_i = \min_j(q_{ij})$
a_1	q_{11}	q_{12}	$\dots$	q_{1n}	α_1
a_2	q_{21}	q_{22}	$\dots$	q_{2n}	α_2
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
a_m	q_{m1}	q_{m2}	$\dots$	q_{mn}	α_m
$\beta_j = \max_i(q_{ij})$	β_1	β_2	$\dots$	β_n	$v_1 = \max_i(\alpha_i)$ / $v_2 = \min_j(\beta_j)$

Tabela 1: Forma generală a unui joc matriceal

2.2.1. Semnificația elementelor matriceale și dualitatea plăților

Fiecare element q_{ij} reprezintă rezultatul numeric (plata) corespunzător intersecției dintre strategia a_i a jucătorului A și strategia b_j a jucătorului B. În ingineria sistemelor și cercetarea operațională, aceste valori nu sunt simple numere, ci cuantifică eficiența unei decizii sub următoarea formă:

- **Câștigul Jucătorului A**: Elementul q_{ij} este beneficiul direct (ex: randament, profit, rată de transmisie). Obiectivul este selectarea unui index i care să maximizeze această valoare, luând în calcul cel mai nefavorabil răspuns al adversarului.
- **Pierderea Jucătorului B**: Conform principiului sumei nule ($f_A + f_B = 0$), același element q_{ij} reprezintă un cost, o penalizare sau un consum de resurse pentru B. Jucătorul B va selecta indexul j care să minimizeze acest impact.

2.2.2. Indicatorii de siguranță α și β (Analiza pe linii și coloane)

Conform algoritmului de rezolvare a jocurilor matriceale, se introduc vectorii de bordură care definesc "atitudinea prudentă" a decidenților:

1. **Vectorul minimelor pe linii (α_i)**: Pentru fiecare strategie a_i, Jucătorul A analizează cel mai mic câștig pe care l-ar putea obține dacă Jucătorul B acționează optim pentru a-l împiedica.
$$\alpha_i = \min\{q_{i1}, q_{i2}, \ldots, q_{in}\} = \min_j q_{ij}$$
2. **Vectorul maximelor pe coloane (β_j)**: Pentru fiecare strategie b_j, Jucătorul B analizează pierderea maximă la care se expune.
$$\beta_j = \max\{q_{1j}, q_{2j}, \ldots, q_{mj}\} = \max_i q_{ij}$$

2.2.3. Valorile critice de prag v_1 și v_2

Sinteza acestor indicatori conduce la determinarea limitelor de stabilitate ale modelului, calculate în celula de colț (dreapta-jos) a matricei prin aplicarea principiului Max-Min:

- **Valoarea inferioară a jocului (v_1)**: Reprezintă pragul de siguranță al Jucătorului A. Este cel mai mare dintre minimele pe linii, garantând un câștig minim de cel puțin v_1, indiferent de ce face B.
$$v_1 = \max_i \alpha_i = \max_i \min_j q_{ij}$$
- **Valoarea superioară a jocului (v_2)**: Reprezintă plafonul pierderilor Jucătorului B. Este cel mai mic dintre maximele pe coloane, garantând că pierderea lui B nu va depăși v_2, indiferent de strategia lui A.
$$v_2 = \min_j \beta_j = \min_j \max_i q_{ij}$$

Determinarea acestor valori constituie etapa preliminară obligatorie. Dacă $v_1 = v_2$, jocul are o soluție stabilă numită Punct de Șa. În caz contrar ($v_1 < v_2$), se demonstrează necesitatea utilizării strategiilor mixte, trecându-se la rezolvarea prin metode de programare liniară (Algoritmul Simplex).

2.3. Analiza strategiilor pure și a variantelor de acțiune

În cadrul unui joc matriceal, o strategie pură reprezintă o decizie deterministă, prin care un jucător alege o singură variantă de acțiune din mulțimea sa de strategii disponibile ($X = \{a_1, a_2, \ldots, a_m\}$ pentru Jucătorul A, respectiv $Y = \{b_1, b_2, \ldots, b_n\}$ pentru Jucătorul B), fără a introduce elemente de probabilitate.

2.3.1. Definirea variantelor de acțiune

Fiecare linie i din matricea Q reprezintă o variantă de acțiune pentru Jucătorul A, iar fiecare coloană j reprezintă o variantă de acțiune pentru Jucătorul B. Alegerea unei strategii pure presupune că:

- Jucătorul A decide să aplice strategia a_i cu probabilitatea $p_i = 1$ (toate celelalte strategii având probabilitatea 0).
- Jucătorul B decide să aplice strategia b_j cu probabilitatea $q_j = 1$.

2.3.2. Mecanismul de selecție în condiții de incertitudine

Deoarece în jocurile matriceale deciziile sunt luate simultan, un inginer sau un decident trebuie să utilizeze un criteriu de alegere care să minimizeze riscul. Analiza strategiilor pure se bazează pe principiul prudenței:

1. **Strategia optimă pentru A**: Este acea variantă a_{i^*} care corespunde valorii maxime din vectorul minimelor pe linii. Aceasta îi garantează lui A cel mai mare câștig posibil în cel mai defavorabil context creat de adversar.
$$i^* \text{ astfel încât } \alpha_{i^*} = \max_i(\min_j q_{ij})$$
2. **Strategia optimă pentru B**: Este acea variantă b_{j^*} care corespunde valorii minime din vectorul maximelor pe coloane, limitând pierderea maximă la care este expus sistemul.
$$j^* \text{ astfel încât } \beta_{j^*} = \min_j(\max_i q_{ij})$$

2.3.3. Aplicabilitate și limitări

Utilizarea strategiilor pure este optimă doar în situația în care jocul posedă un Punct de Șa (echilibru stabil). În inginerie, acest lucru se traduce printr-un regim de funcționare în care nicio parte nu are interesul să își modifice unilateral decizia, deoarece orice schimbare ar duce la un rezultat mai slab.

Dacă analiza indică faptul că $v_1 < v_2$, strategiile pure devin instabile și ușor de anticipat de către adversar. În acest caz, varianta de acțiune optimă se mută în domeniul strategiilor mixte, unde decidentul alternează între strategiile pure conform unor ponderi calculate prin metode de programare liniară (Algoritmul Simplex), subiect ce va fi detaliat în capitolele următoare.

3. Criterii de Optimalitate și Echilibrul Jocului

3.1. Principiile Max-Min și Min-Max: determinarea valorilor v_1 și v_2

În teoria jocurilor matriceale, adoptarea unei decizii se bazează pe principiul prudenței. Acest concept presupune că fiecare jucător este rațional și acționează astfel încât să își protejeze interesele în cel mai nefavorabil context generat de adversar. Acest raționament conduce la definirea criteriilor Max-Min (pentru jucătorul A) și Min-Max (pentru jucătorul B).

3.1.1. Determinarea valorii inferioare a jocului (v_1)

Jucătorul A, având rolul de a maximiza beneficiul net, analizează mai întâi pierderile minime sau „câștigurile cele mai mici" pe care le-ar putea obține pentru fiecare linie a matricei Q. Acest indicator, notat cu α_i, reprezintă limita inferioară de siguranță pentru fiecare strategie a_i:

$$\alpha_i = \min_j q_{ij}$$

Principiul Max-Min presupune alegerea acelei strategii care oferă cel mai bun rezultat dintre toate situațiile defavorabile identificate. Astfel, se determină valoarea inferioară a jocului, v_1 (sau $\underline{v}$), care garantează că jucătorul A va obține un câștig de cel puțin această valoare, indiferent de contra-strategia aleasă de B:

$$v_1 = \max_i \alpha_i = \max_i \left(\min_j q_{ij} \right)$$

3.1.2. Determinarea valorii superioare a jocului (v_2)

Simetric, jucătorul B urmărește minimizarea propriei pierderi (sau a costului de sistem). Pentru fiecare coloană j a matricei, el identifică pierderea maximă la care se expune în cazul în care jucătorul A ar ghici intenția sa, valoare notată cu β_j:

$$\beta_j = \max_i q_{ij}$$

Principiul Min-Max constă în selectarea acelei strategii care minimizează aceste riscuri maxime. Valoarea rezultată, v_2 (sau $\overline{v}$), reprezintă plafonul superior al jocului, asigurându-l pe jucătorul B că pierderea sa nu va depăși acest prag, indiferent de acțiunile jucătorului A:

$$v_2 = \min_j \beta_j = \min_j \left(\max_i q_{ij} \right)$$

3.1.3. Semnificația inegalității fundamentale

O teoremă fundamentală în teoria jocurilor demonstrează că între cele două valori de prag există întotdeauna relația de ordine:

$v_1 \leq v_2$

Această inegalitate are o interpretare critică în inginerie și optimizare:

- Dacă $v_1 < v_2$: Jocul este instabil în strategii pure. Există un „interval de incertitudine" între ceea ce A poate garanta că va câștiga și ceea ce B poate garanta că va pierde. În acest caz, soluția optimă trebuie căutată în domeniul strategiilor mixte.

- Dacă $v_1 = v_2$: Sistemul atinge un punct de echilibru perfect. Această valoare comună $v = v_1 = v_2$ reprezintă valoarea jocului, iar strategiile care conduc la acest rezultat sunt considerate optime și stabile.

3.2. Punctul de Șa (Saddle Point): condiții de existență și stabilitate

Conceptul de Punct de Șa reprezintă fundamentul soluțiilor stabile în jocurile matriceale finite. Denumirea provine din analogia cu suprafețele matematice tridimensionale unde un punct este, simultan, un maxim local pe o direcție și un minim local pe direcția perpendiculară.

3.2.1. Definiție și Condiția de Existență

Din punct de vedere al teoriei jocurilor, un element $q_{i_0 j_0}$ al matricei Q este un punct de șa dacă acesta îndeplinește simultan două condiții de optimitate locală:

1. Este cel mai mic element de pe linia sa (i_0).
2. Este cel mai mare element de pe coloana sa (j_0).

Condiția necesară și suficientă pentru ca un joc de două persoane cu sumă nulă să admită un echilibru în strategii pure este egalitatea dintre valorile de prag determinate anterior:

$$v_1 = v_2 = v$$

unde v este definită ca valoarea jocului. Dacă această egalitate este satisfăcută, perechea de strategii (a_{i_0}, b_{j_0}) constituie soluția optimă a modelului.

3.2.2. Inegalitatea de Stabilitate (Criteriul de Echilibru)

Stabilitatea unui punct de șa este demonstrată prin faptul că niciun jucător nu are un stimulent să își modifice unilateral strategia. Dacă (i_0, j_0) este un punct de șa, atunci pentru orice $i \in \{1, \ldots, m\}$ și orice $j \in \{1, \ldots, n\}$, este valabilă dubla inegalitate:

$$q_{i,j_0} \leq q_{i_0,j_0} \leq q_{i_0,j}$$

Această relație are următoarele implicații decizionale:

- $q_{i,j_0} \leq q_{i_0,j_0}$: Jucătorul A observă că, dacă B își menține alegerea j_0, orice altă strategie i îi va aduce un câștig mai mic sau egal cu cel oferit de i_0.
- $q_{i_0,j_0} \leq q_{i_0,j}$: Jucătorul B observă că, dacă A își menține aleg$q_{i,j_0} \leq q_{i_0,j_0}$erea i_0, orice altă strategie j îi va genera o pierdere mai mare sau egală cu cea oferită de j_0.

3.2.3. Proprietăți ale Punctelor de Șa

În analiza sistemelor complexe, punctele de șa prezintă proprietăți care facilitează rezolvarea numerică:

- Echivalența valorică: Un joc poate poseda mai multe puncte de șa (elemente aflate în poziții diferite), însă toate vor avea obligatoriu aceeași valoare numerică v.

- Interschimbabilitatea: Dacă (i_1, j_1) și (i_2, j_2) sunt puncte de șa, atunci și perechile (i_1, j_2) și (i_2, j_1) sunt puncte de șa. Aceasta înseamnă că decidenții pot alege oricare dintre strategiile optime fără a afecta echilibrul sistemului.
- Absența echilibrului: În situația în care $v_1 < v_2$, se spune că jocul are o lacună de stabilitate. În acest caz, orice alegere a unei strategii pure poate fi contracarată eficient de adversar, forțând trecerea la utilizarea strategiilor mixte (probabilistice).

3.2.4. Exemplu numeric de identificare a Punctului de Șa

Pentru a ilustra procesul de identificare a echilibrului, considerăm următoarea matrice de câștig Q bordată cu indicatorii de minim și maxim:

A \B	b_1	b_2	b_3	$\alpha_i = \min_j(q_{ij})$
a_1	1	4	7	1
a_2	5	6	8	5
a_3	3	1	2	1
$\beta_j = \max_i(q_{ij})$	5	6	8	v = 5

Analiza rezultatelor:

- Minimele pe linii sunt $\{1, 5, 1\}$, deci $v_1 = \max(1, 5, 1) = 5$.
- Maximele pe coloane sunt $\{5, 6, 8\}$, deci $v_2 = \min(5, 6, 8) = 5$.
- Deoarece $v_1 = v_2$, elementul $q_{21} = 5$ este punct de șa. Strategiile optime sunt a_2 pentru jucătorul A și b_1 pentru jucătorul B.

3.3. Teorema lui Nash: generalizarea echilibrului în sistemele decizionale

Atunci când analiza unui joc matriceal indică o lacună de stabilitate ($v_1 < v_2$), strategiile pure devin insuficiente pentru a descrie un comportament optim. În acest context, contribuția lui John Nash a fost revoluționară, demonstrând că orice joc finit posedă cel puțin un punct de echilibru, cu condiția extinderii spațiului decizional.

3.3.1. Trecerea la strategii mixte

O strategie mixtă reprezintă o distribuție de probabilitate peste mulțimea strategiilor pure. Dacă Jucătorul A are strategiile $\{a_1, a_2, \ldots, a_m\}$, o strategie mixtă este un vector $X = (x_1, x_2, \ldots, x_m)$ unde $\sum x_i = 1$ și $x_i \geq 0$.

Teorema lui Nash afirmă că, în orice joc cu un număr finit de jucători și strategii, există cel puțin o configurație de strategii mixte (numită Echilibru Nash) astfel încât niciun jucător nu își poate îmbunătăți câștigul așteptat prin schimbarea unilaterală a propriei strategii.

3.3.2. Semnificația Teoremei în Jocurile cu Sumă Nulă

În cazul particular al jocurilor de două persoane cu sumă nulă (obiectul prezentului proiect), Teorema lui Nash se identifică cu Teorema Minimax a lui von Neumann. Aceasta garantează că:

1. Există întotdeauna o valoare unică a jocului v.
2. Există vectori de probabilitate optimi X^* și Y^* care satisfac relația:
$$E(X, Y^*) \leq E(X^*, Y^*) \leq E(X^*, Y)$$
unde E reprezintă speranța matematică a câștigului (media ponderată a elementelor matricei Q).

3.3.3. Implicații practice în inginerie

Din punct de vedere ingineresc, Teorema lui Nash elimină predictibilitatea. Dacă un sistem (Jucătorul A) ar folosi o singură strategie pură într-un joc fără punct de șa, un atacator sau un mediu advers (Jucătorul B) ar putea exploata această regularitate.

Prin utilizarea strategiilor mixte, decidentul își "maschează" intențiile, alternând variantele de acțiune astfel încât să obțină un randament mediu constant (v), indiferent de reacția adversarului. Determinarea riguroasă a acestor probabilități x_i și y_j nu se mai poate face prin simplă observare, fiind necesară transpunerea problemei într-un model de Programare Liniară, subiect ce va fi tratat în capitolul următor.

4. Relația dintre Teoria Jocurilor și Programarea Liniară

4.1. Transpunerea jocurilor de strategie în modele de optimizare liniară

Atunci când un joc matriceal nu posedă un punct de șa ($v_1 < v_2$), jucătorii trebuie să adopte strategii mixte pentru a-și maximiza câștigul mediu, respectiv pentru a-și minimiza pierderea medie. Această problemă de determinare a probabilităților optime poate fi modelată matematic sub forma unei probleme de programare liniară.

4.1.1. Modelul Jucătorului A (Maximizarea câștigului)

Fie $X = (x_1, x_2, \ldots, x_m)$ vectorul strategiilor mixte ale Jucătorului A, unde x_i reprezintă probabilitatea de a alege strategia pură a_i. Condiția fundamentală este ca $\sum_{i=1}^{m} x_i = 1$, cu $x_i \geq 0$.

Jucătorul A dorește să determine acele valori x_i care să îi garanteze un câștig mediu cel puțin egal cu v, indiferent de strategia b_j aleasă de adversar. Aceasta conduce la următorul sistem de inecuații:
$$\sum_{i=1}^{m} q_{ij} x_i \geq v, \quad j = 1, 2, \ldots, n$$
Pentru a transforma acest model într-o formă canonică de programare liniară, se împart inecuațiile la v (presupunând $v > 0$) și se notează $u_i = \frac{x_i}{v}$. Problema devine:

- Funcția obiectiv: $$[\max] v \iff [\min] \frac{1}{v} = \sum_{i=1}^{m} u_i$$

- Restricții:

$$\begin{cases} \sum_{i=1}^{m} q_{ij} u_i \geq 1, & j = 1, \ldots, n \\ u_i \geq 0, & i = 1, \ldots, m \end{cases}$$

4.1.2. Modelul Jucătorului B (Minimizarea pierderii)

Simetric, Jucătorul B folosește vectorul de probabilități $Y = (y_1, y_2, \ldots, y_n)$ pentru a se asigura că pierderea sa medie nu depășește valoarea v:

$$\sum_{j=1}^{n} q_{ij} y_j \leq v, \quad i = 1, 2, \ldots, m$$

Prin substituția $w_j = \dfrac{y_j}{v}$, modelul pentru Jucătorul B devine:

$$[\min] v \iff [\max] \frac{1}{v} = \sum_{j=1}^{n} w_j$$

- Funcția obiectiv:
- Restricții:

$$\begin{cases} \sum_{j=1}^{n} q_{ij} w_j \leq 1, & i = 1, \ldots, m \\ w_j \geq 0, & j = 1, \ldots, n \end{cases}$$

4.1.3. Avantajele transpunerii

Această metodă de modelare transformă un conflict strategic într-un algoritm computațional riguros. Prin utilizarea programării liniare, se obțin următoarele avantaje:

1. Garantarea soluției: Conform teoremei fundamentale a programării liniare, dacă regiunea admisibilă este mărginită, există întotdeauna o soluție optimă.
2. Eficiență: Jocurile de dimensiuni mari ($m, n > 3$), imposibil de rezolvat grafic, pot fi procesate rapid prin metode numerice.
3. Corelarea Duală: Cele două modele (al lui A și al lui B) formează un cuplu de probleme duale, ceea ce înseamnă că rezolvarea uneia oferă automat și soluția celeilalte.

4.2. Dualitatea în contextul jocurilor: relația dintre strategiile jucătorilor

Conceptul de dualitate în programarea liniară oferă o interpretare economică și strategică esențială pentru teoria jocurilor. Orice joc matriceal cu sumă nulă poate fi privit ca un cuplu de probleme de optimizare (Primală și Dualăă), unde soluția optimă a uneia furnizează în mod intrinsec soluția celeilalte.

4.2.1. Corespondența dintre Primal și Dual

În contextul modelării prezentate la punctul 4.1, dacă definim problema Jucătorului B (minimizarea pierderii) ca fiind Problema Primală, atunci problema Jucătorului A (maximizarea câștigului) reprezintă Problema Dualăă. Această relație este guvernată de următoarele proprietăți de simetrie:

- Matricea coeficienților: Matricea plăților Q din problema primală devine matricea transpusă Q^T în problema duală.
- Vectorul termenilor liberi: Restricțiile unitare din primal (≤ 1) corespund coeficienților funcției obiectiv din dual.
- Sensul optimizării: O problemă de maximizare a beneficiului este întotdeauna cuplată cu o problemă de minimizare a costului asociat.

4.2.2. Teorema Fundamentală a Dualității și Echilibrul

Conform Teoremei Fundamentale a Dualității, dacă una dintre probleme admite o soluție optimă finită, atunci și cealaltă admite o soluție optimă, iar valorile funcțiilor obiectiv în punctul de optim sunt egale:

$$[\max] \sum_{j=1}^{n} w_j = [\min] \sum_{i=1}^{m} u_i = \frac{1}{v}$$

Această egalitate confirmă matematic Teorema Minimax: într-un joc cu sumă nulă, câștigul maxim pe care Jucătorul A îl poate obține prin strategii mixte este exact egal cu pierderea minimă pe care Jucătorul B o poate asigura. Această valoare comună v reprezintă punctul de echilibru al sistemului în sensul lui Nash.

4.2.3. Interpretarea strategiilor prin variabilele duale

Relația dintre strategiile celor doi jucători nu este doar una de opoziție, ci și de condiționare reciprocă. Variabilele duale (umbre) oferă informații despre sensibilitatea sistemului:

1. Strategii active: Dacă o restricție din modelul Jucătorului A este satisfăcută cu semnul "egal" la optim, înseamnă că strategia corespunzătoare a Jucătorului B este o strategie activă (va fi utilizată cu o probabilitate $y_j > 0$).
2. Strategii dominate: Dacă o strategie oferă un randament net inferior valorii jocului (v), variabila duală asociată va fi zero, indicând faptul că acea variantă de acțiune nu face parte din mixul optim de decizii (este o strategie dominată).

Prin utilizarea dualității, efortul computațional este redus considerabil. Algoritmul Simplex, aplicat pe una dintre forme (de regulă cea care are mai puține restricții), oferă simultan:

- Vectorul X^*: Strategiile mixte optime pentru Jucătorul A.
- Vectorul Y^*: Strategiile mixte optime pentru Jucătorul B.
- Valoarea v: Pragul de rentabilitate al interacțiunii.

4.3 Strategii de calcul numeric și selecția algoritmului optim

După ce jocul a fost transpus în limbajul programării liniare, pasul final constă în alegerea metodei de calcul care să ofere soluția cu cel mai mic efort computațional. Deși teoretic ambele variante ale algoritmului Simplex (Primal și Dual) conduc la același punct de echilibru Nash, în practica inginerească se face o alegere bazată pe structura restricțiilor.

4.3.1. Justificarea utilizării Algoritmului Simplex Primal (ASP)

Optăm pentru Algoritmul Simplex Primal (ASP) ca metodă principală de rezolvare, în detrimentul variantei duale (ASD), din următoarele considerente practice:

1. Simplitatea punctului de start: Modelul Jucătorului B, având toate restricțiile de tipul "≤ 1", ne oferă un avantaj major: originea este deja o soluție admisibilă. Acest lucru înseamnă că algoritmul poate începe imediat căutarea optimului fără a fi nevoie de

faze intermediare complicate (cum ar fi Variabilele Artificiale sau Metoda celor două faze).

2. Stabilitatea în implementarea software: Majoritatea bibliotecilor de calcul numeric (precum cele din Python, Matlab sau Excel Solver) utilizează varianta Primală ca standard. Alegând ASP.
3. Monitorizarea progresului: În ASP, valoarea funcției obiectiv crește constant spre valoarea maximă la fiecare iterație. Acest aspect este util pentru un decident, deoarece oferă o estimare a modului în care mixul de strategii se apropie de echilibrul final.

4.3.2. Rolul Dualității în interpretarea soluției

Deși calculul propriu-zis se realizează prin ASP, conceptul de Simplex Dual rămâne extrem de valoros pentru analiza post-optim:

- Valorile Duale: În tabelul final de calcul, pe linia funcției obiectiv, vom regăsi "prețurile umbră" sau variabilele duale. Acestea reprezintă exact probabilitățile de joc pentru Jucătorul A.
- Validarea Echilibrului: Rezolvând o singură dată problema prin ASP, obținem "la pachet" și soluția problemei duale, confirmând astfel că beneficiul maxim al lui A este egal cu pierderea minimă a lui B.

Concluzie: Alegerea ASP reprezintă calea cea mai scurtă și mai robustă de la matricea jocului la cifrele concrete ale probabilităților, transformând o problemă strategică abstractă într-o soluție numerică gata de aplicat în mediul real.

5. Implementarea Computațională a Modelului

5.1 Implementarea algoritmului

Algoritmul pentru rezolvarea Teoriei Jocurilor a fost implementat în limbajul de programare Python (versiunea 3.13), utilizând următoarele biblioteci și module:

- customtkinter (versiunea 5.2.2)
- darkdetect (versiunea 0.8.0)
- numpy (versiunea 2.4.3)
- packaging (versiunea 26.0)

Aplicația respectă principiile programării orientate pe obiecte, fiind structurată pe clase și module separate pentru interfață, logică și validări.

Complexitatea algoritmului implementat este de ordinul $O(n \cdot m)$, unde n reprezintă numărul de restricții, iar m numărul de variabile, datorită operațiilor matriciale).

5.2 Codul sursa

5.2.1 Algoritmul pentru tabelul simplex

```
import customtkinter as ctk
from utils.conversie_Fractie import transformaraFractie
```

```
# Tabel Simplex
class tabelSimplex(ctk.CTkFrame):

    DARK_PURPLE = "#1c103c"
    MEDIUM_PURPLE = "#5f2b6e"
    LIGHT_MAGENTA = "#c95ea8"
    WHITE = "#ffffff"

    def __init__(self, container, titlu, matrice, coloana_xb, costuri, deltas, baza,
nume_variabile, pivot=None,
                 iteratie=0):
        super().__init__(container, border_width=2, border_color=self.DARK_PURPLE,
fg_color="#271c46")
        self.pack(pady=10, padx=10, fill="x")

        # titlul tabelului
        ctk.CTkLabel(
            self, text=titlu, font=("Poppins", 14, "bold"),
            text_color=self.WHITE, fg_color="transparent"
        ).pack(pady=5)

        # zona unde desenam tabelul simplex
        f_grila = ctk.CTkFrame(self, fg_color="transparent")
        f_grila.pack(pady=5)

        nr_linii, nr_coloane = len(baza), len(costuri)

        CULOARE_CRUCE = self.MEDIUM_PURPLE
        CULOARE_PIVOT = self.LIGHT_MAGENTA
        CULOARE_NEGATIV = "#FF5252"
        CULOARE_POZITIV = "#2ECC71"

        # Eticheta MAX
        ctk.CTkLabel(
            f_grila, text="MAX", width=75,
            font=("Poppins", 12, "bold"), text_color=self.WHITE, fg_color="transparent"
        ).grid(row=0, column=2, padx=1, pady=1)

        # antet variabile
        for j in range(nr_coloane):
            ctk.CTkLabel(
                f_grila, text=nume_variabile[j], width=75,
                font=("Poppins", 12, "bold"), text_color=self.WHITE, fg_color="#160d2e"
```

```
        ).grid(row=0, column=3 + j, padx=1, pady=1)

    # Cj
    ctk.CTkLabel(f_grila, text="Cj", width=75, font=("Poppins", 12, "bold"),
text_color=self.WHITE,
                 fg_color="#160d2e").grid(row=1, column=2, padx=1, pady=1)
    for j, val in enumerate(costuri):
        fundal = CULOARE_CRUCE if (pivot and j == pivot[1]) else "#160d2e"
        ctk.CTkLabel(f_grila, text=transformaraFractie(val), width=75, font=("Poppins", 12,
"bold"),
                     text_color=self.WHITE, fg_color=fundal).grid(row=1, column=3 + j, padx=1,
pady=1)

    # antet stanga
    for j, text_antet in enumerate(["CB", "B", "XB"]):
        ctk.CTkLabel(
            f_grila, text=text_antet, width=75,
            font=("Poppins", 12, "bold"), text_color=self.WHITE, fg_color="#160d2e"
        ).grid(row=2, column=j, padx=1, pady=1)

    # linii tabel
    for i in range(nr_linii):
        idx_variabila_baza = baza[i]
        fundal_linie = CULOARE_CRUCE if (pivot and i == pivot[0]) else
self.DARK_PURPLE
        text_color_linie = self.WHITE if (pivot and i == pivot[0]) else self.WHITE

        ctk.CTkLabel(f_grila, text=transformaraFractie(costuri[idx_variabila_baza]),
width=75,
                     fg_color=fundal_linie, text_color=text_color_linie).grid(row=3 + i,
column=0, padx=1, pady=1)
        ctk.CTkLabel(f_grila, text=nume_variabile[idx_variabila_baza], width=75,
fg_color=fundal_linie,
                     text_color=text_color_linie).grid(row=3 + i, column=1, padx=1, pady=1)
        ctk.CTkLabel(f_grila, text=transformaraFractie(coloana_xb[i]), width=75,
fg_color=fundal_linie,
                     text_color=text_color_linie).grid(row=3 + i, column=2, padx=1, pady=1)

        for j in range(nr_coloane):
            culoare_celula = self.DARK_PURPLE
            text_col = self.WHITE

            if pivot:
                if i == pivot[0] and j == pivot[1]:
```

```
                    culoare_celula = CULOARE_PIVOT
                    text_col = "#ffffff"
                elif i == pivot[0] or j == pivot[1]:
                    culoare_celula = CULOARE_CRUCE
                    text_col = "#ffffff"

            font_celula = ("Poppins", 12, "bold") if (pivot and i == pivot[0] and j == pivot[1])
else (
                "Poppins", 12)

            ctk.CTkLabel(
                f_grila, text=transformaraFractie(matrice[i, j]), width=75,
                fg_color=culoare_celula, text_color=text_col, font=font_celula
            ).grid(row=3 + i, column=3 + j, padx=1, pady=1)

        # Calcul Zj si Z0
        cb = [costuri[idx] for idx in baza]
        z_j = [sum(cb[i] * matrice[i, j] for i in range(nr_linii)) for j in range(nr_coloane)]
        z_0 = sum(cb[i] * coloana_xb[i] for i in range(nr_linii))

        rand_zj = 3 + nr_linii
        rand_delta = 4 + nr_linii

        text_z = f"Z^{iteratie} = {transformaraFractie(z_0)}"
        ctk.CTkLabel(
            f_grila, text=text_z, font=("Poppins", 12, "bold"), text_color=self.WHITE
        ).grid(row=rand_zj, column=1, columnspan=2, padx=1, pady=(5, 1))

        for j, z_val in enumerate(z_j):
            fundal_zj = CULOARE_CRUCE if (pivot and j == pivot[1]) else
self.DARK_PURPLE
            text_color_zj = "#ffffff" if (pivot and j == pivot[1]) else self.WHITE
            ctk.CTkLabel(
                f_grila, text=transformaraFractie(z_val),
                font=("Poppins", 12, "bold"), text_color=text_color_zj,
                fg_color=fundal_zj, width=75
            ).grid(row=rand_zj, column=3 + j, padx=1, pady=(5, 1))

        # Delta
        ctk.CTkLabel(
            f_grila, text="Δj", font=("Poppins", 12, "bold"), text_color=self.WHITE
        ).grid(row=rand_delta, column=1, columnspan=2, pady=(1, 5))

        for j, d_val in enumerate(deltas):
```

```
        culoare_text = CULOARE_POZITIV if d_val <= 1e-9 else CULOARE_NEGATIV
        fundal_delta = CULOARE_CRUCE if (pivot and j == pivot[1]) else
self.DARK_PURPLE

        ctk.CTkLabel(
            f_grila, text=transformaraFractie(d_val),
            font=("Poppins", 12, "bold"), text_color=culoare_text,
            fg_color=fundal_delta, width=75
        ).grid(row=rand_delta, column=3 + j, padx=1, pady=(1, 5))
```

5.2.2. Algoritmul in PSEUDOCOD

Intrare: matricea de plată Q cu m linii și n coloane
Ieșire: valoarea jocului V, strategiile optime X^0 și Y^0

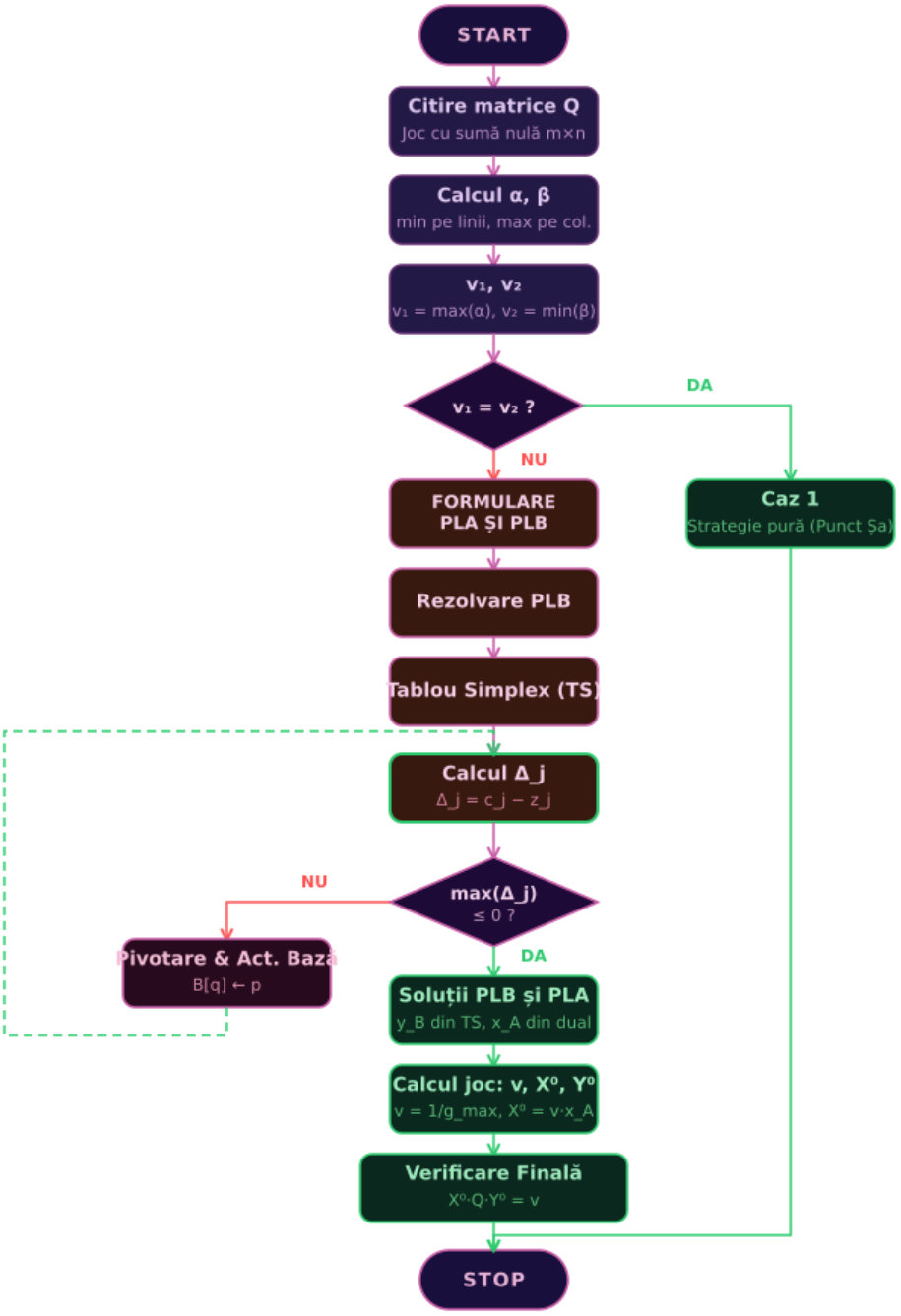
```
citește m, n
citește Q  /* matricea m × n */

pentru i ← 1, m execută
| αi ← min(Q[i][1], Q[i][2], …, Q[i][n])
└■

pentru j ← 1, n execută
| β□ ← max(Q[1][j], Q[2][j], …, Q[m][j])
└■

v1 ← max(α1, α2, …, α□)
v2 ← min(β1, β2, …, β□)

dacă v1 = v2 atunci
| /* Caz 1 – punct de tip "șa", strategie pură */
| i* ← indicele i pentru care αi = v1
| j* ← indicele j pentru care β□ = v2
| pentru i ← 1, m execută
| | dacă i = i* atunci  X⁰[i] ← 1  altfel  X⁰[i] ← 0
| └■
| pentru j ← 1, n execută
| | dacă j = j* atunci  Y⁰[j] ← 1  altfel  Y⁰[j] ← 0
| └■
| V ← v1
| scrie "CAZ 1 – Strategie pură:  V =", V
| scrie "X⁰ =", X⁰, "   Y⁰ =", Y⁰
| stop
└■

dacă min(Q) ≤ 0 atunci
| k ← 1 − min(Q)
```

```
altfel
| k ← 0
└■
Q' ← Q + k   /* translatare pentru a asigura Q' > 0 */

/* Formulare PLB: max Σy□  s.t.  Q'ᵀ·y ≤ 1,  y ≥ 0
   Forma standard cu variabile slack s₁…s□:
     mat = [Q' | I□],  XB = (1,1,…,1),  Baza = {s₁,…,s□}  */
pentru i ← 1, m execută
| pentru j ← 1, n execută
| | mat[i][j] ← Q'[i][j]
| └■
| pentru j ← 1, m execută
| | dacă i = j atunci  mat[i][n+j] ← 1  altfel  mat[i][n+j] ← 0
| └■
| XB[i] ← 1
| Baza[i] ← n + i
└■
pentru j ← 1, n   execută  C[j]   ← 1  └■
pentru j ← 1, m   execută  C[n+j] ← 0  └■

it ← 0
repetă
| /* Calcul Δ□ = c□ − cB · A□ */
| pentru j ← 1, n+m execută
| | delta[j] ← C[j]
| | pentru i ← 1, m execută
| | | delta[j] ← delta[j] − C[Baza[i]] · mat[i][j]
| | └■
| └■
|
| dacă max(delta) ≤ 0 atunci
| | scrie "Soluție optimă la iterația", it
| | stop   /* ieși din repetă */
| └■
|
| /* Coloana pivot */
| p ← indicele j cu delta[j] maxim
|
| /* Linia pivot – testul raportului minim */
| pentru i ← 1, m execută
| | dacă mat[i][p] > 0 atunci
| | | r[i] ← XB[i] / mat[i][p]
| | altfel
| | | r[i] ← +∞
| | └■
| └■
| q ← indicele i cu r[i] minim
```

```
|
| /* Operația de pivotare */
| XB[q] ← XB[q] / mat[q][p]
| pentru j ← 1, n+m execută
| | mat[q][j] ← mat[q][j] / mat[q][p]
| └■
| pentru i ← 1, m execută
| | dacă i ≠ q atunci
| | | XB[i] ← XB[i] − mat[i][p] · XB[q]
| | | pentru j ← 1, n+m execută
| | | | mat[i][j] ← mat[i][j] − mat[i][p] · mat[q][j]
| | | └■
| | └■
| └■
| Baza[q] ← p
| it ← it + 1
până când max(delta) ≤ 0

pentru j ← 1, n execută
| yBaza[j] ← 0
└■
pentru i ← 1, m execută
| dacă Baza[i] ≤ n atunci
| | yBaza[Baza[i]] ← XB[i]
| └■
└■
pentru i ← 1, m execută
| xDual[i] ← −delta[n+i]   /* soluție duală din deltele slack */
└■

gmax    ← Σ yBaza[j]  /* pentru j = 1 … n */
vTransf ← 1 / gmax
V       ← vTransf − k

pentru i ← 1, m execută
| X⁰[i] ← vTransf · xDual[i]
└■
pentru j ← 1, n execută
| Y⁰[j] ← vTransf · yBaza[j]
└■

scrie "V =", V
scrie "X⁰ =", X⁰,  "  (Σ X⁰ = 1 ✓)"
scrie "Y⁰ =", Y⁰,  "  (Σ Y⁰ = 1 ✓)"
scrie "Verificare:  X⁰ · Q · Y⁰ =", X⁰ · Q · Y⁰, " = V ✓"
```

SFÂRȘIT ALGORITM

5.3 Schema logica


START
Citire matrice Q
Joc cu sumă nulă m×n
Calcul α, β
min pe linii, max pe col.
v₁, v₂
v₁ = max(α), v₂ = min(β)
v₁ = v₂ ?
DA
NU
FORMULARE PLA ȘI PLB
Caz 1
Strategie pură (Punct Șa)
Rezolvare PLB
Tablou Simplex (TS)
Calcul Δ_j
Δ_j = c_j − z_j
NU
max(Δ_j) ≤ 0 ?
Pivotare & Act. Bază
B[q] ← p
DA
Soluții PLB și PLA
y_B din TS, x_A din dual
Calcul joc: v, X⁰, Y⁰
v = 1/g_max, X⁰ = v·x_A
Verificare Finală
X⁰·Q·Y⁰ = v
STOP

6. Aplicație Practică: "Duelul Strategic"

6.1. Construcția modelului decizional și a matricei de câștig

Contextul Decizional:

Se analizează un scenariu competitiv de tip „joc cu sumă nulă” între doi agenți economici (Jucătorul A și Jucătorul B). Aceștia trebuie să aleagă simultan o strategie de acțiune dintr-un set de trei variante posibile, într-un mediu caracterizat prin incertitudine și interdependență strategică.

Variantele de Acțiune (Strategii Pure):

Fiecare jucător are la dispoziție următoarele opțiuni tactice:

1. S_1 (Atac rapid): Vizibilitate imediată pe piață, dar cu un impact de penetrare scăzut.
2. S_2 (Atac puternic): Impact strategic masiv, dar cu o latență ridicată (consum mare de timp și resurse).
3. S_3 (Apărare): Strategie de conservare, axată pe protecția segmentului actual de piață.

Datele Problemei (Matricea Câștigurilor Q):

Interacțiunea dintre alegerile celor doi jucători generează un rezultat numeric (puncte de cotă de piață) care se regăsește în matricea plăților de mai jos. Valorile reprezintă câștigul Jucătorului A, care este egal cu pierderea Jucătorului B:

$$Q = \begin{pmatrix} 2 & 4 & 1 \\ 1 & 3 & 5 \\ 3 & 2 & 4 \end{pmatrix}$$

Obiectivele Studiului de Caz (Cerințe):

În cadrul acestui sistem de modelare și optimizare (SMO), se solicită parcurgerea următoarelor etape:

1. Analiza Stabilității Deterministe: Determinarea valorilor de siguranță α_i (minimele pe linii) și β_j (maximele pe coloane).
2. Verificarea Existenței Punctului de Șa: Identificarea valorilor de prag v_1 (max-min) și v_2 (min-max) pentru a stabili dacă jocul admite o soluție stabilă în strategii pure.
3. Modelarea în Programare Liniară: În cazul în care $v_1 < v_2$, să se construiască modelele duale (Primal și Dual) pentru determinarea probabilităților de aplicare a fiecărei strategii.
4. Determinarea Soluției Optime: Calcularea vectorilor de probabilitate X^* și Y^*, precum și a valorii jocului (v), utilizând principiile algoritmului Simplex.

5. Interpretarea Strategică: Formularea unei recomandări decizionale bazate pe rezultatele matematice obținute.

6.2. Determinarea soluțiilor optime și interpretarea rezultatelor

Obiectivul acestei secțiuni este aplicarea algoritmului de rezolvare pentru jocuri matriceale, parcurgând etapele de analiză a strategiilor pure și, ulterior, a celor mixte.

6.2.1. Analiza stabilității în strategii pure (Criteriul Max-Min / Min-Max)

Primul pas în determinarea soluției constă în calcularea indicatorilor de siguranță pe linii și coloane pentru matricea Q:

A \ B	b_1	b_2	b_3	$\alpha_i = \min_j q_{ij}$
a_1	2	1	3	1
a_2	4	3	2	2
a_3	1	5	4	1
$\beta_j = \max_i q_{ij}$	4	5	3	$v_1 = \max_i \alpha_i = 2$ / $v_2 = \min_j \beta_j = 3$

Valoarea inferioară a jocului (v_1): Reprezintă câștigul minim garantat pentru Jucătorul A.
$v_1 = \max_i(\alpha_i) = \max(1, 1, 2) = 2$

Valoarea superioară a jocului (v_2): Reprezintă pierderea maximă acceptată de Jucătorul B.
$v_2 = \min_j(\beta_j) = \min(3, 4, 5) = 3$

Interpretare: Deoarece $v_1 < v_2$ ($2 < 3$), jocul prezintă o lacună de stabilitate. Nu există un „Punct de Șa”, ceea ce înseamnă că nicio strategie pură nu poate constitui un echilibru stabil; orice alegere fixă a unui jucător poate fi exploatată de adversar dacă acesta o anticipează.

Pas 2: Atașarea problemelor de Programare Liniară (PLA și PLB)

Pentru determinarea strategiilor mixte, definim modelele duale de optimizare:

PLA (MIN)

- $\min f(x) = x_1 + x_2 + x_3$
- $\begin{cases} 2x_1 + 1x_2 + 3x_3 \geq 1 \\ 4x_1 + 3x_2 + 2x_3 \geq 1 \\ 1x_1 + 5x_2 + 4x_3 \geq 1 \end{cases}$
- $x_{1,2,3} \geq 0$

PLB (MAX)

- $\max g(y) = y_1 + y_2 + y_3$
- $\begin{cases} 2y_1 + 4y_2 + 1y_3 \leq 1 \\ 1y_1 + 3y_2 + 5y_3 \leq 1 \\ 3y_1 + 2y_2 + 4y_3 \leq 1 \end{cases}$
- $y_{1,2,3} \geq 0$

Pas 3: Alegem sa rezolvam PLB (MAX)

MAX			y1↓	y2	y3	s1	s2	s3
CB	B	XB	1	1	1	0	0	0
0	s_1	1	2	4	1	1	0	0
0	s_2	1	1	3	5	0	1	0
0	$s_3 \uparrow$	1	[3]	2	4	0	0	1
		$z^0 = 0$	$\Delta_1 = 1$	$\Delta_2 = 1$	$\Delta_3 = 1$	$\Delta_4 = 0$	$\Delta_5 = 0$	$\Delta_6 = 0$

Tabelul I_1:

Pivotul este coloana y_2 ($\Delta_2 = 1/3$). Raportul minim:
$\min\{(1/3)/(8/3), (2/3)/(7/3), (1/3)/(2/3)\} = \min\{1/8, 2/7, 1/2\} = 1/8$. Pivotul este 8/3.

CB	B	XB	y1	y2↓	y3	s1	s2	s3
0	$s_1 \uparrow$	1/3	0	[8/3]	-5/3	1	0	-2/3
0	s_2	2/3	0	7/3	11/3	0	1	-1/3
1	y_1	1/3	1	2/3	4/3	0	0	1/3
		$z^1 = 1/3$	$\Delta_1 = 0$	$\Delta_2 = 1/3$	$\Delta_3 = -1/3$	$\Delta_4 = 0$	$\Delta_5 = 0$	$\Delta_6 = -1/3$

Tabelul I_2 (Final):

Toți Δ_j de pe linia de jos sunt $\Delta_j \leq 0$. STOP.

CB	B	XB	y1	y2	y3	s1	s2	s3
1	y_2	1/8	0	1	-5/8	3/8	0	-1/4
0	s_2	3/8	0	0	41/8	-7/8	1	1/4
1	y_1	1/4	1	0	7/4	-1/4	0	1/2
		$z^2 = 3/8$	$\Delta_1 = 0$	$\Delta_2 = 0$	$\Delta_3 = -1/8$	$\Delta_4 = -1/8$	$\Delta_5 = 0$	$\Delta_6 = -1/4$

Pas 4: Extragerea și Interpretarea Soluției Finale

După ce Tabelul Simplex I_2 a indicat stop (toți $\Delta_j \leq 0$), extragem coordonatele punctului de echilibru.

4.1. Valoarea Jocului (v) – Indicatorul de Performanță

Valoarea funcției obiectiv la optim în modelul nostru de maximizare a fost $z^2 = 3/8$. În teoria jocurilor matriceale, valoarea jocului (v) este inversul acestei sume:

$$v = \frac{1}{z^2} = \frac{1}{3/8} = \frac{8}{3} \approx 2.66$$

Ce reprezintă v?

Aceasta este "cota de piață" sau punctajul mediu pe care Jucătorul A îl va obține în mod constant, indiferent de cât de bine joacă Jucătorul B, atâta timp cât A își respectă strategia optimă. Este un progres semnificativ față de valoarea de siguranță pură ($v_1 = 2$).

4.2. Soluția Jucătorului A (X_{opt}) – Strategia de Maximizare

Jucătorul A este cel care „pândește” din umbră în tabelul de maximizare al lui B. Soluția lui se află pe linia funcției obiectiv (Δ), sub coloanele variabilelor de ecart (s_1, s_2, s_3):

1. Extragerea variabilelor duale (u):
   - $u_1 = \Delta_4 = 1/8$ (corespunde strategiei a_1)
   - $u_2 = \Delta_5 = 0$ (corespunde strategiei a_2)
   - $u_3 = \Delta_6 = 1/4$ (corespunde strategiei a_3)
2. Transformarea în probabilități ($X_{opt} = v \cdot u$):
   - $P(a_1) = \frac{8}{3} \cdot \frac{1}{8} = \mathbf{\frac{1}{3}} \approx \mathbf{33.3\%}$
   - $P(a_2) = \frac{8}{3} \cdot 0 = \mathbf{0\%}$
   - $P(a_3) = \frac{8}{3} \cdot \frac{1}{4} = \mathbf{\frac{2}{3}} \approx \mathbf{66.6\%}$

Interpretare tactică A: Jucătorul A trebuie să renunțe complet la "Atacul Puternic" (a_2). Deși pare tentant, acesta îl expune prea mult. Succesul lui A vine dintr-o combinație de Atac Rapid (1/3 din timp) și Apărare (2/3 din timp).

4.3. Soluția Jucătorului B (Y_{opt}) – Strategia de Minimizare

Soluția Jucătorului B este directă și se citește din coloana Termenilor Liberi (X_B) a tabelului final:

1. Extragerea variabilelor primale (y):
- $y_1 = 1/4$ (corespunde strategiei b_1)
- $y_2 = 1/8$ (corespunde strategiei b_2)
- $y_3 = 0$ (variabila y_3 nu a intrat în bază, deci este 0)
2. Transformarea în probabilități ($Y_{opt} = v \cdot y$):
   - $P(b_1) = \frac{8}{3} \cdot \frac{1}{4} = \mathbf{\frac{2}{3}} \approx \mathbf{66.6\%}$
   - $P(b_2) = \frac{8}{3} \cdot \frac{1}{8} = \mathbf{\frac{1}{3}} \approx \mathbf{33.3\%}$

- $P(b_3) = \frac{8}{3} \cdot 0 = \mathbf{0}\%$

Interpretare tactică B: Jucătorul B ignoră complet "Apărarea" (b_3). El alege să fie agresiv pentru a limita câștigurile lui A, folosind predominant Atacul Rapid (2/3 din timp) și ocazional Atacul Puternic (1/3 din timp).

Pas 5: Verificarea Soluției Optime prin Calcul Matricial

Pentru a valida rezultatele obținute prin algoritmul Simplex, utilizăm relația fundamentală a echilibrului în teoriile jocurilor: $X^0 \cdot Q \cdot Y^0 = V$.

1. Definirea elementelor de calcul

- Vectorul Jucătorului A: $X^0 = \begin{pmatrix} 1/3 & 0 & 2/3 \end{pmatrix}$
- Matricea Câștigurilor: $Q = \begin{pmatrix} 2 & 4 & 1 \\ 1 & 3 & 5 \\ 3 & 2 & 4 \end{pmatrix}$
- Vectorul Jucătorului B: $Y^0 = \begin{pmatrix} 2/3 \\ 1/3 \\ 0 \end{pmatrix}$
- Valoarea Jocului: $V = 8/3$

2. Calculul Etapei I: Produsul $X^0 \cdot Q$

Înmulțim vectorul linie al Jucătorului A cu matricea Q pentru a vedea câștigurile ponderate pe fiecare coloană:

- Coloana 1: $(1/3 \cdot 2) + (0 \cdot 1) + (2/3 \cdot 3) = 2/3 + 0 + 6/3 = \mathbf{8/3}$
- Coloana 2: $(1/3 \cdot 4) + (0 \cdot 3) + (2/3 \cdot 2) = 4/3 + 0 + 4/3 = \mathbf{8/3}$
- Coloana 3: $(1/3 \cdot 1) + (0 \cdot 5) + (2/3 \cdot 4) = 1/3 + 0 + 8/3 = 9/3 = \mathbf{3}$

Rezultatul intermediar este vectorul: $\begin{pmatrix} 8/3 & 8/3 & 3 \end{pmatrix}$.

3. Calculul Etapei II: Produsul Final cu Y^0

Înmulțim vectorul rezultat anterior cu vectorul coloană al Jucătorului B pentru a obține speranța matematică finală:

$$\begin{pmatrix} 8/3 & 8/3 & 3 \end{pmatrix} \cdot \begin{pmatrix} 2/3 \\ 1/3 \\ 0 \end{pmatrix} = (8/3 \cdot 2/3) + (8/3 \cdot 1/3) + (3 \cdot 0)$$

- Termenul 1: $16/9$
- Termenul 2: $8/9$
- Suma: $16/9 + 8/9 = 24/9$

Simplificăm fracția prin 3:

$24/9 = \mathbf{8/3}$

Concluzia Verificării

Prin calcul direct, am demonstrat că:

$8/3 = 8/3 \, (\text{Adevărat})$

Interpretare:

Această egalitate confirmă faptul că strategiile mixte identificate (X^0 și Y^0) sunt optime. Niciun jucător nu își poate îmbunătăți câștigul/pierderea prin modificarea unilaterală a frecvenței acțiunilor sale. Verificarea a fost realizată cu succes, validând întregul model de optimizare liniară aplicat în studiul de caz.

7. Concluzii

Proiectul „Duelul Strategic” a demonstrat că optimizarea deciziilor într-un mediu competitiv necesită o abordare matematică riguroasă, transformând un conflict subiectiv într-un model de calcul precis. Prin parcurgerea etapelor de analiză, am validat utilitatea Cercetărilor Operaționale în gestionarea incertitudinii strategice.

Prima etapă a evidențiat că strategiile pure sunt insuficiente atunci când jocul prezintă o lacună de stabilitate ($v_1 < v_2$). Lipsa punctului de șa în matricea plăților a forțat tranziția către strategiile mixte, singura modalitate de a elimina predictibilitatea și de a împiedica adversarul să exploateze o decizie fixă.

Punctul central al lucrării a fost utilizarea Algoritmului Simplex Primal (ASP). Prin transpunerea jocului în Programare Liniară, am demonstrat dualitatea dintre câștigul maxim al Jucătorului A și pierderea minimă a Jucătorului B. Rezultatele numerice au indicat o valoare optimă a jocului de 2.66 puncte, o îmbunătățire semnificativă față de pragul de siguranță inițial.

Aplicația practică a scos în evidență importanța eliminării strategiilor dominate. Deși „Atacul Puternic” (a_2) părea tentant prin prisma câștigului maxim, optimizarea a demonstrat că succesul real provine dintr-un mix stabil de Atac Rapid (33%) și Apărare (66%). Această combinație asigură un randament constant pe termen lung.

În final, verificarea matricială a confirmat atingerea Echilibrului Nash, punctul în care niciun jucător nu își mai poate îmbunătăți situația prin schimbări unilaterale. Proiectul confirmă astfel că utilizarea sistemelor de modelare și optimizare (SMO) oferă stabilitate și un avantaj strategic decisiv, transformând confruntarea într-un sistem controlat și eficient.

8. Bibliografie

1. Bibic Simona Mihaela, *Cercetări Operaționale, suport de curs electronic, https://archive.curs.upb.ro/2024/course/view.php?id=9543.*
2. Nash, J. F. – *Equilibrium Points in n-Person Games*, Proceedings of the National Academy of Sciences, 1950.
3. Neumann, J. von, & Morgenstern, O. – *Theory of Games and Economic Behavior*, Princeton University Press, 1944..
4. Taha, H. A. – *Operations Research: An Introduction*, 10th Edition, Pearson Education, 2017.
5. Văduva, I. – *Modele de simulare și optimizare*, Editura Universității din București, 2018.
6. Nisan, N., Roughgarden, T., Tardos, É., & Vazirani, V. V. (Eds.). (2007). Algorithmic Game Theory. Cambridge University Press.
7. https://cloudconvert.com/png-to-ico
8. https://www.rapidtables.com/math/symbols/Basic_Math_Symbols.html
9. https://numpy.org
10. https://docs.python.org/3/library/fractions.html
11. https://mermaid.live
12. https://docs.python.org/3/
13. https://www.w3schools.com/python/default.asp
14. https://customtkinter.tomschimansky.com